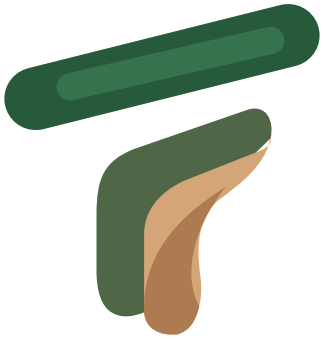


TaskFlow

A Unified Project and Task Management Platform with Time Tracking and Client Collaboration

Project Proposal

Team Members

ID	Name
1	Mostafa Mohamed Mahmoud Mohamed *(Team Leader)*
2	Ahmed Reda Ahmed Sedik
3	Alaa Essam Hamed Abdelkader
4	Ahd Ammar Eissa Elbakry
5	Eman Mohamed Mosaad Elmetboly

Program	DEPI
Project Domain	Project Planning & Management
Version	2.0
Status	Draft for team and instructor review
Date	October 14, 2026

Table of Contents

1. Executive Summary 2

2. Market Size 2

3. Problem & Market Validation 3

3.1 The Problem 3
3.2 Why It Matters Now 3
3.3 Who Feels This Problem? 3
3.4 Market Validation Plan 4
3.5 User Validation Groups 4
3.6 Problem Severity 4

4. Core TaskFlow Functionality 5

4.1 Core Platform Features 5
4.2 Why a Unified Workflow Is Needed 6
4.3 Model Choice & Solution Fit (Non-AI) 6

5. Data Strategy & Feature Validation 6

5.1 Data Managed by TaskFlow 6
5.2 Data Sources 7
5.3 Validating Time Tracking and Progress 7
5.4 Validation Against User Expectations 7

6. Technical Architecture 8

6.1 How the Application Will Be Built 8
6.2 How the Main Modules Fit Together 8
6.3 Security, Reliability, and Real-Time Updates 9

7. Business Feasibility 9

7.1 Target Audience 9
7.2 Market Need and Relevance 10
7.3 Competitor Analysis 10
7.4 Business Model 10
7.5 Initial Pricing Validation 11
7.6 Technical and Operational Feasibility 11
7.7 Overall Feasibility 12

8. Scope and Exclusions 12

9. Project Objectives and Success Criteria 13

10. Expected Deliverables 13

11. Assumptions and Constraints 13

12. References 14

Project at a Glance

Idea: Web-based project and task management platform for individuals, freelancers, teams, and companies.
Target: Individuals, freelancers, small teams, and service agencies working with clients.
Core edge: One workflow from private planning to time tracking and client deliverable review.
Stack: ASP.NET Core MVC, EF Core, SQL Server / Azure SQL, SignalR. No AI/ML features.

1. Executive Summary

TaskFlow is a web-based project and task management platform designed for individuals, freelancers, small teams, and companies. It helps users organize projects, break work into manageable tasks, plan schedules, collaborate with other people when needed, and track progress in one place.

The platform combines interactive Kanban boards, Gantt-style schedule visualization, task and subtask management, comments, file attachments, notifications, activity history, and reporting. In addition, TaskFlow is planned to include an optional client portal, a project live room, a work timer, deliverable review, change requests, work handover notes, and rule-based project health alerts.

TaskFlow is not limited to organizations. A single user can start with a private workspace and personal projects. A freelancer can manage separate projects for different clients, track work sessions, and selectively share approved progress or deliverables. A team can invite members and divide responsibilities, while a company can organize multiple workspaces and projects with role-based access.

The proposed technology stack includes ASP.NET Core MVC, Entity Framework Core, SQL Server or Azure SQL, ASP.NET Core Identity, SignalR, Bootstrap, JavaScript, and supporting libraries for Kanban, Gantt charts, and analytics. The application will use conventional programming logic, validation, scheduled jobs, and rule-based automation. **AI, machine learning, and LLM-based functionality are explicitly out of scope.**

2. Market Size

Project management software is an established global software category serving organizations and professionals that need to plan, coordinate, and track work. Grand View Research estimates that the global project management software market was valued at **USD 7.9 billion in 2025**, estimates **USD 8.7 billion for 2026**, and forecasts it could reach **USD 19.8 billion by 2033**, representing an estimated compound annual growth rate of 11.1% for 2026–2033. These are third-party market estimates, not TaskFlow revenue forecasts; see the source in References.

This broad market includes products with different target users, price points, and feature sets. It should not be treated as TaskFlow's directly addressable market. TaskFlow's initial serviceable audience is narrower: individuals, freelancers, small teams, and service businesses that need project organization, time tracking, and optional client visibility.

A reliable Egypt-specific market-size estimate has not yet been established for this proposal. Before claiming a local TAM/SAM/SOM figure, the team should gather credible country-level data and define its assumptions. A practical first-stage market assessment will therefore focus on user interviews, a small survey, competitor feature comparisons, and willingness-to-pay testing rather than presenting an unsupported local market number.

3. Problem & Market Validation

3.1 The Problem

Work is often spread across task lists, spreadsheets, email, chat messages, file storage, calendars, and separate time-tracking tools. Users may struggle to answer basic questions: What needs to be done next? Who owns a task? What is delayed? How much time was spent? Which deliverables need approval? What did the client request to change?

For freelancers and small service providers, the problem also includes keeping separate client projects organized, sharing progress without exposing private work notes, and maintaining a clear record of deliverables, review decisions, and scope changes.

For teams, unclear responsibilities and dependencies can delay work. For clients, limited visibility can lead to repeated status requests and uncertainty about what has been completed or what still needs approval.

TaskFlow aims to address these issues by connecting planning, execution, time tracking, delivery review, and optional client collaboration through a consistent project workflow.

3.2 Why It Matters Now

Distributed and hybrid collaboration makes clear, accessible project information useful for both individuals and teams. Freelancers may handle multiple clients at the same time; small teams may need lightweight coordination without adopting a complex enterprise process; clients increasingly expect understandable progress updates and reviewable deliverables.

However, these observations are problem hypotheses, not proof of demand for TaskFlow specifically. The team must validate which user groups experience these problems most severely and which features would meaningfully improve their current workflow.

3.3 Who Feels This Problem?

- **Individuals and students:** Need a straightforward way to organize personal, study, and portfolio projects without setting up a team.
- **Freelancers:** Need to separate clients and projects, record work sessions, monitor deadlines, and share agreed information with each client.
- **Small teams:** Need task ownership, shared boards, deadlines, comments, work handover, and real-time status updates.
- **Project managers:** Need to monitor schedules, hours, dependencies, overdue tasks, reviews, and project risks.

- **Clients:** Need controlled access to progress, deliverables, feedback, and approvals without access to the team's private workspace.
- **Small service companies and agencies:** Need a repeatable process for delivering projects to multiple clients and managing change requests.

3.4 Market Validation Plan

At the time of writing, no completed user interviews or survey results have been provided for this proposal. Therefore, no interview outcomes are presented as established facts. The team will validate the problem before drawing conclusions about demand.

Proposed validation activities:

1. Conduct short interviews with individual users, freelancers, project managers, and clients.
2. Ask participants how they currently organize tasks, track time, share progress, review deliverables, and handle changes.
3. Identify which tools they use today, what they like about them, and where work is duplicated or lost.
4. Test low-fidelity screens for the personal dashboard, task timer, client portal, and deliverable review workflow.
5. Run a small survey to measure feature importance, current tool usage, and willingness to try a new workflow.
6. Record findings, sample size, participant group, questions, and limitations in the Requirements Gathering documentation.

Validation should focus on observed workflows and concrete examples rather than asking only whether participants “like” the idea.

3.5 User Validation Groups

Freelancer validation: Confirm whether tracking hours by task/project, separating client data, and selectively sharing progress would solve recurring problems. Determine whether a simple timer is enough or whether approval and billable-time reports are required.

Team and project-manager validation: Confirm whether Kanban, schedule visibility, task dependencies, handover notes, and rule-based project alerts are helpful, and whether the workflow is too complex for small teams.

Client validation: Check which information clients actually need, how they prefer to review deliverables, what approval decisions they must make, and what project data must remain private.

3.6 Problem Severity

The team will rank each problem by frequency, impact, and the current workaround. A proposed interview scoring method is a 1–5 scale for each factor:

- **Frequency:** How often does the issue occur?
- **Impact:** How much time, cost, confusion, or delay does it cause?
- **Workaround burden:** How difficult is the current workaround?

The score can help prioritize features, but it must be based on gathered evidence and should not be interpreted as a statistically representative market result when the sample is small.

4. Core TaskFlow Functionality

4.1 Core Platform Features

Accounts and workspaces

- Registration, login, account management, and role-based access control.
- A private workspace for individuals, with optional shared workspaces for teams.
- Workspace membership and project-level access rules.

Project and task management

- Create, edit, archive, and restore projects.
- Kanban boards with configurable columns, drag-and-drop, priorities, labels, due dates, assignees, subtasks, checklists, and task dependencies.
- Gantt-style timeline visualization for schedules, milestones, and task dependencies.
- Search, filters, saved views, comments, attachments, and activity history.

Time tracking

- Start, pause, resume, and stop a timer associated with a task.
- Persist work sessions so a refresh or browser closure does not automatically lose the recorded session.
- Summarize recorded time by user, task, project, and selected time period.
- Compare estimates with actual recorded time.
- Keep recorded work time distinct from approved billable time; billing is not part of the initial commitment.

Client collaboration

- Optional client accounts and a client portal limited to explicitly shared projects and data.
- A project live room with approved progress information, milestones, shared discussion, and deliverables.
- Deliverable versions and review decisions: approved, changes requested, or pending review.
- Change requests that record the requested scope change, expected schedule/cost impact, and approval decision.

Team coordination and project oversight

- Work handover notes for completed work, remaining work, known blockers, and next steps.
- Rule-based project health alerts for conditions such as overdue critical tasks, long-running blockers, unusually high actual time, or deliverables waiting too long for review.
- Notifications and activity logs that explain important changes and decisions.

4.2 Why a Unified Workflow Is Needed

TaskFlow's proposed value is not that it invents every project-management feature independently. Rather, it aims to provide a consistent workflow across personal work, client work, and team collaboration. Users can start alone and enable collaboration only when needed, without having to adopt an organization-first setup immediately.

The connected workflow is:

1. Create a project and organize tasks.
2. Assign responsibility and plan dates.
3. Track progress through Kanban and schedule views.
4. Record actual work sessions against tasks.
5. Prepare a deliverable and make it available for review.
6. Record client feedback, approval, or requested changes.
7. Reassess dates and workload when scope changes are approved.
8. Review reports and project-health alerts.

The usefulness of this integrated experience must be validated through the planned interviews and prototype tests.

4.3 Model Choice & Solution Fit (Non-AI)

TaskFlow will use deterministic software logic instead of AI. Examples include:

- Status transitions governed by configured project workflow rules.
- Project-health alerts triggered by explicit thresholds and conditions.
- Timer durations calculated from persisted start, pause, resume, and stop events.
- Progress percentages calculated from defined task or milestone rules.
- Permission checks based on the user's identity, workspace membership, project access, and role.
- Automation rules triggered by known application events or scheduled jobs.

This approach fits the project because its central tasks are structured records, workflow transitions, timing, permissions, and transparent rules. Users should be able to understand why a status, alert, or calculated value appears. AI and machine learning are not required and will not be included in this project version.

5. Data Strategy & Feature Validation

5.1 Data Managed by TaskFlow

TaskFlow is expected to store the following categories of data:

- Account and profile data.
- Workspaces, workspace memberships, teams, and permissions.
- Projects, boards, columns, tasks, subtasks, checklists, labels, and dependencies.

- Time entries containing a user, task, start/end times, pause intervals or equivalent duration records, and edit history as required.
- Comments, attachments metadata, deliverables, deliverable versions, reviews, and client feedback.
- Change requests, approval decisions, handover notes, notifications, and activity logs.
- Aggregated reporting data calculated from the underlying records.

The database should store file metadata and access controls. Uploaded files should be stored using a suitable file or object-storage mechanism rather than placing large file contents directly into relational database rows by default.

5.2 Data Sources

The primary data source is information entered or generated during normal product use: projects created by users, task updates, recorded time sessions, review decisions, and explicit collaboration events. External data will not be scraped or assumed to be available.

Market context will be drawn from cited public industry research and official competitor documentation. User needs will be assessed using interviews, surveys, and prototype feedback conducted by the team. Any collected research data should be limited to what is necessary and reported honestly, including sample size and limitations.

5.3 Validating Time Tracking and Progress

Timer correctness is central to the intended user experience. Testing should cover:

- Start, pause, resume, and stop operations.
- Refreshing or closing the browser while a timer is active.
- Preventing accidental duplicate active timers if the rule is one timer per user.
- Handling interrupted sessions and user-requested corrections.
- Ensuring that a session is counted once and is associated with the correct user and task.
- Summing time by task, user, project, and reporting period.
- Restricting who may view, edit, approve, or export time entries.

Project progress also needs an explicit calculation definition. For example, a simple first version may calculate progress as completed tasks divided by total eligible tasks. The team must document how archived, cancelled, weighted, or blocked tasks affect the calculation rather than assuming every project uses the same rule.

5.4 Validation Against User Expectations

Feature validation will compare actual system behavior with documented acceptance criteria and participant feedback. The team should verify that:

- Users understand task status and project-progress calculations.
- A freelancer can use the core workflow without inviting a team.
- A client can only access explicitly shared projects and deliverables.
- Review decisions and change requests preserve an auditable history.
- Work-time reports match the saved time entries.

- Alerts fire only when their configured conditions are met.

No model training or model accuracy evaluation is needed because the project does not use AI. Test evidence will consist of test cases, measured results, defect reports, and user feedback.

6. Technical Architecture

6.1 How the Application Will Be Built

The proposed stack is:

- **Backend and web UI:** ASP.NET Core MVC and C#.
- **Persistence:** Entity Framework Core.
- **Database:** SQL Server or Azure SQL, depending on the approved environment.
- **Authentication:** ASP.NET Core Identity.
- **Real-time communication:** SignalR.
- **Frontend:** Razor Views, HTML, CSS, JavaScript, and Bootstrap 5.
- **Kanban interaction:** Dragula.js or a team-approved equivalent.
- **Gantt visualization:** Frappe Gantt or a team-approved equivalent.
- **Charts:** Chart.js.
- **Version control and collaboration:** Git and GitHub.
- **Hosting and monitoring:** Microsoft Azure if access, cost, and project constraints permit.

The solution is planned as a layered application:

- **TaskFlow.Web:** Controllers, views, presentation logic, validation feedback, and user interaction.
- **TaskFlow.Services:** Business rules for projects, tasks, timers, client access, deliverable reviews, change requests, handover, and alerts.
- **TaskFlow.Data:** EF Core context, database mappings, migrations, and data access/repositories where useful.

Dependency Injection will connect components. Repository and Unit of Work patterns will be used only where they add clear value; EF Core's DbContext already provides change tracking and transaction-related capabilities. The application will use SOLID principles, centralized error handling, logging, and automated tests for key business rules.

6.2 How the Main Modules Fit Together

A typical task update follows this flow:

1. A user performs an action in the web interface.
2. The controller validates the request and identifies the current user.
3. The service verifies permissions and business rules.
4. The data layer saves the change, using a transaction where several related records must remain consistent.

5. An activity record is written for events that require an audit trail.
6. SignalR notifies authorized connected users when a real-time update is appropriate.
7. Notification and reporting views reflect the persisted result.

For example, when a developer stops a timer, the backend must validate the active time session and save the resulting time entry. The server—not a client-side clock alone—must remain the authoritative source for saved session state. If a deliverable is approved, the review record must identify the deliverable version, reviewer, decision, and timestamp.

6.3 Security, Reliability, and Real-Time Updates

- Enforce authorization on the server for projects, tasks, attachments, time entries, and client-visible data.
- Check workspace and project membership on every relevant request; hiding a button in the interface is not sufficient protection.
- Validate user input and uploaded files; use safe file storage and controlled download access.
- Protect against common web vulnerabilities, including XSS, CSRF, and injection attacks.
- Handle concurrent task edits and timer actions safely.
- Scope SignalR groups and events so users receive only updates they are authorized to see.
- Store secrets in environment configuration or a secure secret store, not in source control.
- Use logs, automated tests, backups, and monitoring appropriate to the hosting environment.

7. Business Feasibility

7.1 Target Audience

TaskFlow is intended for four overlapping groups:

1. **Individuals and students** managing personal, academic, and portfolio work.
2. **Freelancers** handling multiple client projects and wanting task-level time tracking and controlled progress sharing.
3. **Small teams** needing shared workflows, assignment, and coordination.
4. **Small service companies and agencies** managing multiple projects, client review cycles, and change requests.

The initial target should be validated and narrowed based on user research. For an academic prototype, freelancers and small project teams may be a practical first focus because their workflows can be tested with a small number of participants.

7.2 Market Need and Relevance

The cited industry research indicates that project management software is an established and growing category. The underlying user problems—organizing work, tracking deadlines, recording effort, and sharing progress—are relevant to the intended audience, but market size alone does not prove that users will adopt TaskFlow.

TaskFlow's proposed positioning is a flexible, non-AI workflow that can start as a private personal workspace and optionally expand to client and team collaboration. This positioning must be tested against existing tools and validated with real user feedback.

7.3 Competitor Analysis

TaskFlow operates in a competitive category. The following comparison is high-level and based on publicly documented product capabilities; features and plan limits may change.

Table 1: Competitor comparison.

Product	Publicly documented strengths relevant to TaskFlow	Implication for TaskFlow
Trello	Visual board/card workflow, task assignment, due dates, collaboration, and a free plan.	TaskFlow needs more than a Kanban board to establish value; time tracking, client review, and connected handover/change workflows may be worth testing.
Jira	Kanban and Scrum boards, work-item tracking, multiple project views, and integrations.	TaskFlow should aim for a simpler entry path for individuals/freelancers while still supporting structured team work.
ClickUp	Tasks, boards, calendar, sprint capabilities, time tracking on paid plans, and broader workspace functions.	Many core features are already available elsewhere, so TaskFlow should validate the value of its integrated personal-to-client workflow.
Teamwork.com	Project views, time tracking, client collaboration, and service-business-oriented reporting/features across plans.	Teamwork is a close comparator for freelancer/agency workflows; TaskFlow must avoid claiming that time tracking or client visibility alone are unique.

Potential differentiation to validate—not a proven competitive advantage—is the coherent flow from private individual work to client-visible delivery, combining recorded time, deliverable versions/reviews, change requests, and task handover in a relatively understandable experience. The project should not claim that no competitor offers these features individually.

Official competitor references are provided in References.

7.4 Business Model

A potential future business model is freemium, with a useful personal tier and paid options for users who need more collaboration or administration. This is a hypothesis for feasibility analysis, not a committed commercial launch plan.

Possible tiers to test:

- **Personal / Free:** Personal projects, Kanban, basic deadlines, and limited time tracking/reporting.

- **Freelancer:** Multiple client projects, client portal, deliverable review, richer time reports, and change requests.
- **Team:** Shared workspaces, expanded collaboration, role-based permissions, team reporting, and configurable workflows.
- **Business:** Advanced controls, audit and export options, greater storage/capacity, and administrative capabilities.

The final feature boundaries should be designed only after a cost estimate and willingness-to-pay research. For the DEPI deliverable, the project can demonstrate the product without implementing subscription billing.

7.5 Initial Pricing Validation

No TaskFlow pricing has been validated. The team should not present a specific price as proven until it has been tested with target users.

A proposed validation approach is to:

1. Interview freelancers and small teams about current tools and whether they pay for them.
2. Ask which features are essential versus optional.
3. Compare hypothetical monthly prices or package options using a short survey.
4. Test whether users would choose a free personal option and pay for client collaboration, advanced reports, or team administration.
5. Record the number and type of participants and treat the result as indicative only if the sample is small.

The academic project may use a free or demonstrative deployment without collecting payments. Subscription management and billing are outside the required initial scope unless DEPI or the team explicitly adds them.

7.6 Technical and Operational Feasibility

Technical feasibility: The proposed stack is compatible with the planned web application. The primary technical challenges are authorization isolation between clients and projects, reliable timer/session handling, real-time updates, versioned deliverables, change approval history, and consistent progress reporting. These can be handled using conventional application architecture and database design.

Operational feasibility: The team will need clear feature ownership, code review, issue tracking, Git branching conventions, shared acceptance criteria, and documented test scenarios. The team should prioritize a working end-to-end flow over implementing every advanced feature at once.

Cost feasibility: Local development can use free development tools. Hosting, storage, email delivery, and any paid cloud capabilities depend on account availability and usage limits. The implementation should be designed so that local demonstration and testing do not depend on paid services.

Timeline feasibility: The DEPI planning baseline runs through final testing, reporting, and presentation on 4 December 2026. The feature scope is broad; therefore, the team should define a minimum viable release and move optional enhancements to later increments if time becomes limited.

7.7 Overall Feasibility

TaskFlow is technically feasible as a layered ASP.NET Core web application, provided the team controls scope and tests high-risk workflows early. The project is educationally valuable because it includes authentication, authorization, relational data modeling, task-state workflows, real-time communication, timer calculations, reporting, client-facing access controls, and automated testing.

Commercial feasibility remains unproven until user interviews, usability testing, competitor comparison, and willingness-to-pay research are completed. For the DEPI project, success should first be defined as delivering a secure, tested, clearly documented prototype that demonstrates the core workflow end to end.

8. Scope and Exclusions

In Scope

- Individual/personal workspace and optional shared team workspaces.
- Projects, customizable Kanban boards, tasks, subtasks, deadlines, priorities, labels, and dependencies.
- Gantt-style schedule view.
- Authentication, workspace/project membership, and server-side authorization.
- Task timer, persisted time entries, and time summaries.
- Comments, attachments, notifications, activity history, and real-time updates.
- Optional client portal and project live room with controlled data sharing.
- Deliverable versions, review decisions, and feedback.
- Change requests, work handover notes, and deterministic project-health alerts.
- Reporting, automated tests, setup documentation, and project demonstration.

Out of Scope for This Version

- AI, machine learning, LLM-based assistants, AI-generated summaries, or AI-driven recommendations.
- Native mobile applications; responsive web design is in scope.
- Full accounting, invoicing, payroll, or payment processing.
- Employee surveillance, screenshots of workers, or automatic activity monitoring.
- Guaranteed third-party integrations unless selected and approved by the team.
- Enterprise-scale advanced portfolio optimization.

9. Project Objectives and Success Criteria

The project aims to:

1. Provide a usable project and task workflow for individuals and collaborative users.
2. Allow users to understand project status through Kanban and schedule views.
3. Record and summarize time sessions accurately.
4. Allow clients to access only the information shared with them.
5. Record deliverable-review decisions and approved change requests with an auditable history.
6. Notify relevant users about configured deadlines and health conditions.
7. Apply server-side validation, secure access control, logging, and tests.
8. Deliver documentation and a reproducible GitHub project suitable for DEPI review.

Measurable targets, measurement definitions, and test methods are documented in the separate `05-project-kpis.md` document. Targets remain proposed until confirmed by the team and instructor.

10. Expected Deliverables

- Project planning documents: proposal, project plan, role/task assignment, risk assessment, and KPIs.
- Requirements documentation: stakeholders, user stories, use cases, and functional/non-functional requirements.
- Analysis and design documentation: architecture, database model, and system diagrams.
- Source code and setup instructions in GitHub.
- A demonstrable web application implementing the agreed scope.
- Test plan, test cases, test results, and defect notes.
- User guide, technical documentation, and final presentation materials.

11. Assumptions and Constraints

- The named team members, final feature ownership, and detailed estimates must be confirmed by the team.
- The DEPI schedule supplied by the team is the planning baseline.
- Market size estimates are third-party estimates and do not prove demand for TaskFlow specifically.
- User validation and pricing validation are planned activities, not completed research, unless the team adds documented evidence.
- Cloud deployment and paid service usage depend on available accounts, cost, and DEPI requirements.
- The final feature list may be prioritized into a core release and optional additions to protect implementation and testing quality.

- AI is excluded from the project scope.

12. References

[1] Grand View Research, *Project Management Software Market Report, 2026–2033*. Market estimates published on the report page. https://www.grandviewresearch.com/industry-analysis/project-management-software-market-report

[2] Atlassian, *What are the different Jira plans?* https://www.atlassian.com/software/jira/guides/more/jira-editions

[3] Trello, *Pricing*. https://trello.com/en/pricing

[4] ClickUp, *Pricing and Plans*. https://clickup.com/pricing

[5] Teamwork.com, *Pricing Plans*. https://www.teamwork.com/pricing/

> **Research note:** Competitor pricing, plan limits, and included features can change. Re-check official pages before final submission. Any interview findings, survey statistics, instructor feedback, and local market estimates must be added only when the team has collected and documented them.